Laboratorio CRUD

Administración de personas | C# y SQL Server

Estudiantes:

- Rafael Flores
- Boris Sánchez
- Roger Torrero
- Fatima Calderón

Introducción

En este laboratorio desarrollé una aplicación de escritorio para administrar personas. La aplicación permite consultar, agregar, editar y eliminar registros guardados en SQL Server. Organicé las consultas en una clase independiente de los formularios para comprender la relación entre la interfaz y la base de datos.

Objetivo

Desarrollar una aplicación CRUD con C#, Windows Forms y ADO.NET, conectada a SQL Server mediante autenticación de Windows y consultas parametrizadas.

Herramientas utilizadas

Utilicé **C# y .NET Framework 4.8** como lenguaje y plataforma; **Windows Forms** para los controles; **Visual Studio Community y MSBuild** para editar y compilar; **SQL Server 2025 Express LocalDB**, instancia `(localdb)\MSSQLLocalDB`, para almacenar datos; y **ADO.NET con System.Data.SqlClient** para ejecutar SQL. Preparé y consulté la base con **sqlcmd**. El mismo script puede ejecutarse en SSMS.

Desarrollo

Creé la base `CrudWindowsForms` y la tabla `dbo.People`. Id es un INT con IDENTITY(1,1) y clave primaria; Name es VARCHAR(100) obligatorio; Age es INT obligatorio. El script incorpora Ana, Luis y Maria al crear la tabla. Puede repetirse sin eliminar datos ni duplicar las filas iniciales.

La clase People representa una persona. PeopleDB contiene cinco métodos directos: Get consulta todas las filas; GetById recupera una persona; Add inserta; Update modifica; Delete elimina. Las lecturas utilizan SqlDataReader y ExecuteReader. Las modificaciones utilizan ExecuteNonQuery. Cada operación abre su propia SqlConnection y libera los recursos mediante using.

En Form1 coloqué el encabezado, la cuadrícula de solo lectura y los botones Nuevo, Editar, Eliminar y Actualizar. El evento Load consulta automáticamente la base. La selección de fila completa identifica el objeto People asociado a la fila, evitando confundir la posición visual con el ID.

Reutilicé FrmNuevo para agregar y editar. Si no recibe un ID, inserta; si recibe un ID, carga la persona y actualiza ese registro. Antes de guardar, valida que el nombre contenga entre 1 y 100 caracteres y que la edad sea un entero entre 0 y 120. Cancelar no ejecuta consultas de modificación. Los mensajes de validación aparecen junto a los campos.

Los parámetros @Name, @Age y @Id se agregan con su tipo SQL. El nombre nunca se concatena dentro de la consulta. UPDATE y DELETE incluyen WHERE Id = @Id. El formulario principal solicita confirmación de eliminación con Sí/No, dejando No como opción predeterminada. Después de guardar o eliminar, vuelve a consultar los datos y muestra el resultado en la barra inferior.

La cadena de conexión está en App.config y usa autenticación de Windows. Para otro equipo se cambia Data Source por la instancia real utilizada en SSMS. No se almacenan contraseñas.

Pruebas realizadas

Fecha de ejecución: **8 de octubre de 2026**, hora local de Panamá. Se ejecutaron **19 comprobaciones de SQL**, **33 comprobaciones de formularios**, **una comprobación de construcción para el diseñador** y **12 comprobaciones interactivas**, todas correctas. Las pruebas emplearon bases exclusivas con nombres únicos, eliminadas al finalizar; la base del laboratorio conserva sus tres registros iniciales.

Caso	Entrada o pasos	Resultado esperado	Resultado obtenido / evidencia
Inicio y conexión	Abrir Form1 con tres filas SQL	Carga automática de ID, nombre y edad	Correcto: 3 filas; pruebas-formularios.txt y 01-principal.png.
Inserción e ID	Guardar O'Connor, 30	Una fila con ID generado	Correcto: una fila; ID mayor que los iniciales.
Nombre vacío	Vacío y tres espacios	Rechazar sin insertar	Correcto; cantidad de filas sin cambios.
Nombre largo	101 caracteres	Rechazar; campo limitado a 100	Correcto: validación y MaxLength comprobados.
Edad inválida	Vacío, abc, -1, 121, 2.5 y 2147483648	Rechazar sin insertar ni cerrar con OK	Correcto en los seis casos; 02-validacion.png.
Límites	Edades 0 y 120; nombre de 100 caracteres	Aceptar	Correcto: validación de formulario y almacenamiento SQL comprobados.
Apóstrofo e inyección	O'Connor y texto parecido a DROP TABLE	Almacenar como datos	Correcto: nombre conservado y tabla existente.
Edición	Cambiar O'Connor a O'Connor editado, edad 31	Mantener ID y otras filas	Correcto: consulta independiente; 04-editar-persona.png.
Cancelación	Cambiar campos y ejecutar Cancelar	Conservar nombre y edad anteriores	Correcto: DialogResult.Cancel y SQL sin cambios.
Eliminar en SQL	Delete del ID de prueba	Eliminar una fila y conservar las demás	Correcto: fila ausente, otras filas intactas.

Confirmación de eliminación	Solicitar confirmación del ID 4 y responder No / Sí	No conserva; Sí elimina solo ID 4	Correcto en interacción: No conservó IDs 4 y 5; Sí eliminó ID 4 y conservó ID 5. Capturas 09 y 10; sql-despues-no.txt y sql-despues-si.txt.

Caso	Entrada o pasos	Resultado esperado	Resultado obtenido / evidencia
Actualizar	Insertar por conexión externa y usar Actualizar	Mostrar nuevas filas	Correcto en prueba automática y mediante el botón: O'Connor y Control aparecen. Captura 08. Se usó SQL directo, no SSMS.
Persistencia	Cerrar y crear otro formulario y conexión; cerrar y reabrir el proceso	Recuperar cambios guardados	Correcto: prueba automática con dato editado y reapertura interactiva que conserva ID 5 y la eliminación del ID 4. Captura 13.
Tabla vacía	Borrar solo filas de pruebas y abrir Form1; pulsar Editar y Eliminar	Tabla sin filas; solicitar selección sin fallar	Correcto: ambos botones muestran Seleccione una persona de la tabla. Capturas 05, 06 y 07.
Registro ausente	GetById, Update y Delete con ID inexistente	null o cero filas afectadas	Correcto; no modifica otras filas.
Compilación	Rebuild Debug y Release	Sin errores	Ambas configuraciones compilaron.
Diseñador	Construir ambos formularios sin App.config	Crear los controles sin conexión SQL	Correcto: pruebas-disenador.txt. La configuración se usa al ejecutar las operaciones.

Conclusión

Aprendí a conectar una interfaz de Windows Forms con SQL Server sin utilizar un ORM. El formulario recoge y valida los datos; PeopleDB ejecuta la consulta parametrizada; SQL Server conserva la información. Consultar nuevamente después de cada cambio permite mostrar los datos persistidos. La separación entre formularios y acceso a datos facilita explicar y comprobar cada operación.

Referencias

• Guía de laboratorio: **CRUD con C#, Windows Forms, SQL Server y ADO.NET**, archivo proporcionado `proy ds iv.pdf`, 25 páginas. Se revisaron los requisitos, las etapas de implementación, las pruebas y el apartado 21.

• Héctor de León. **Crear CRUD en C# .NET Windows Forms, SQL Server y ADO.NET | Sin EntityFramework**. https://www.youtube.com/watch?v=hV7G3uYb6AI&t=30s. El enlace se intentó consultar; el servicio web limitó el acceso. La guía proporcionada reconstruye el procedimiento del video. No se afirma haber reproducido el video durante la validación.

Evidencias de funcionamiento

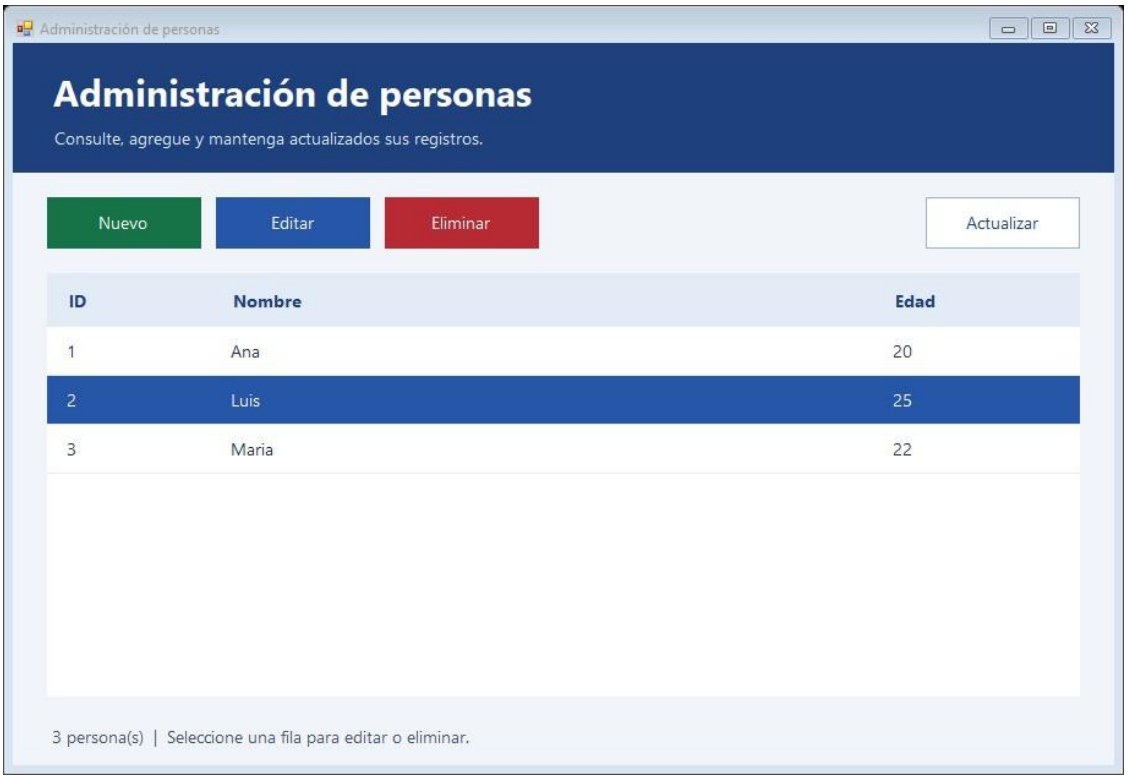


Figura 1. Formulario real durante las pruebas automáticas: tres registros de SQL Server y selección de fila completa.

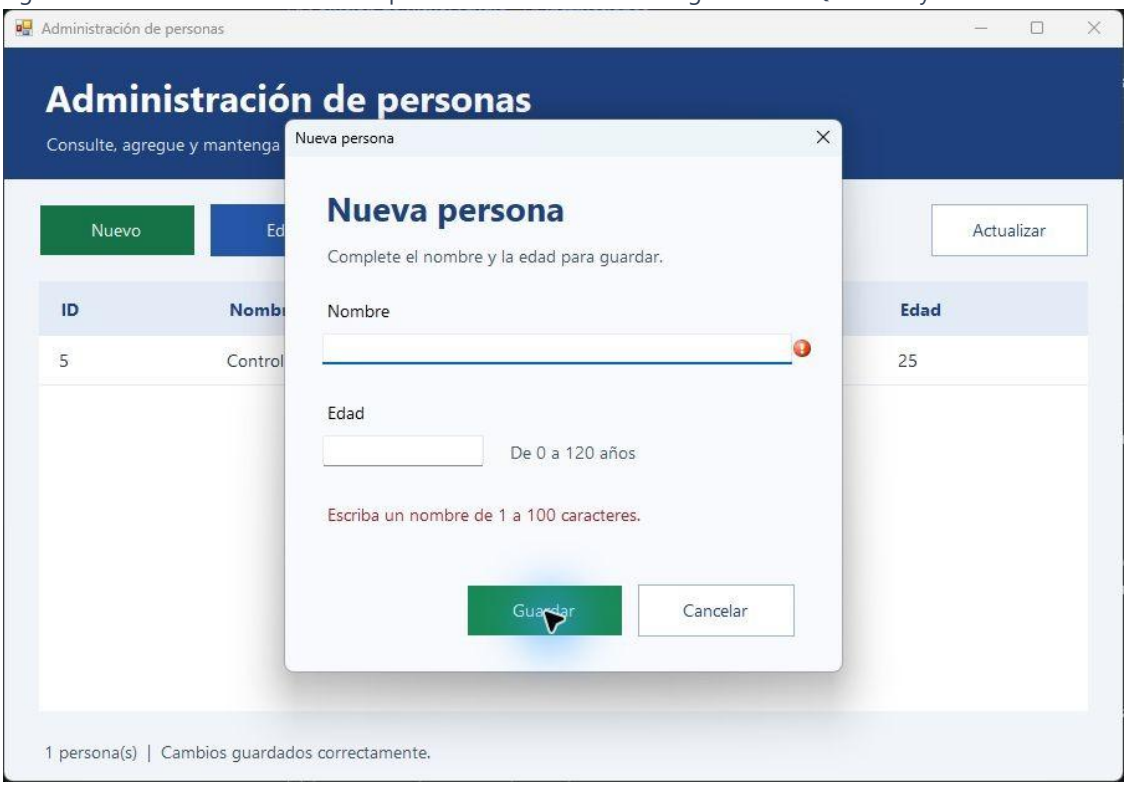


Figura 2. Captura de la aplicación abierta: Guardar rechaza un nombre vacío y mantiene abierto el formulario.

Confirmación y persistencia

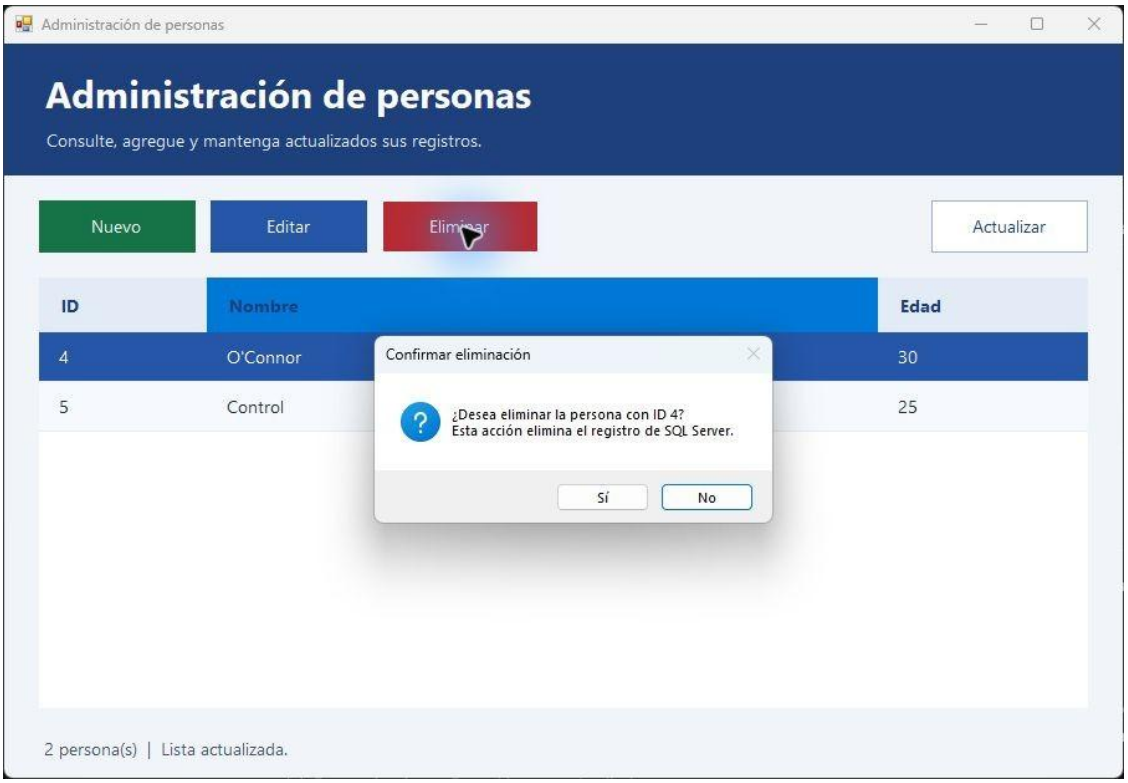


Figura 3. Captura real del diálogo Sí/No. La opción No está seleccionada de forma predeterminada; el ID se muestra antes de eliminar.

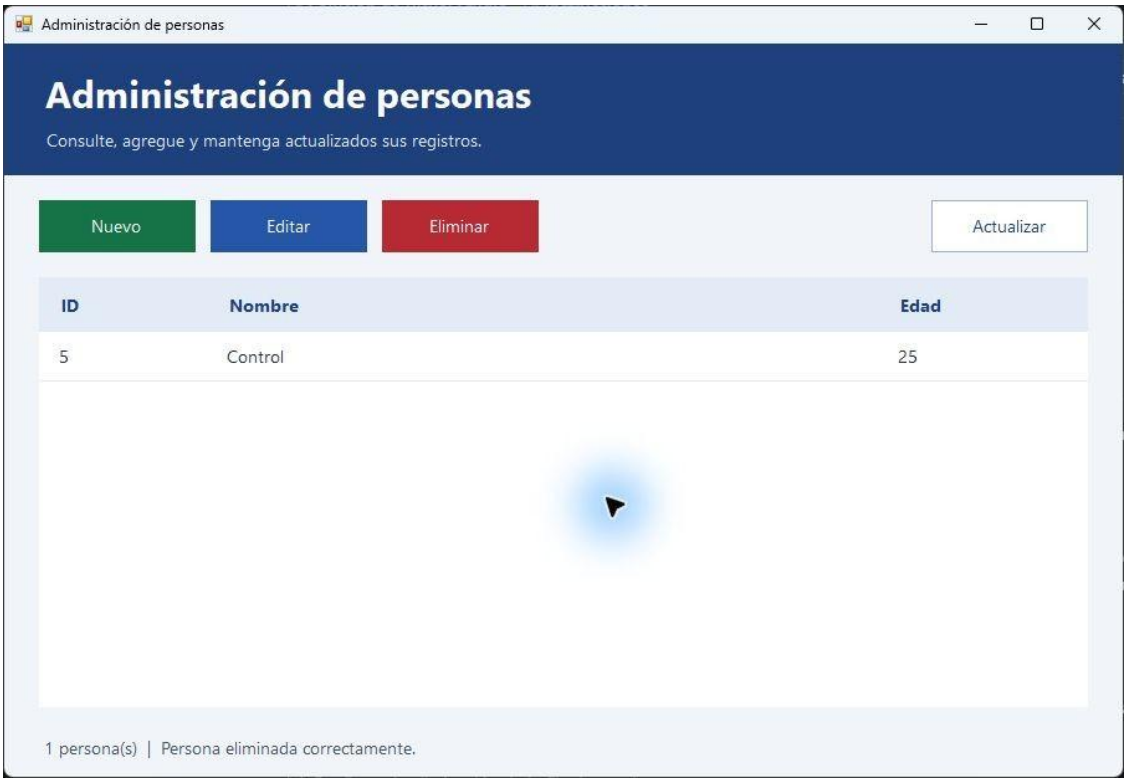


Figura 4. Después de confirmar Sí, se eliminó el ID 4 y se conservó Control (ID 5). El SELECT independiente produjo el mismo resultado.

Respuestas al apartado 21

1. **¿Cuáles son las cuatro operaciones CRUD y qué instrucción SQL corresponde a cada una?** Crear usa INSERT; consultar usa SELECT; actualizar usa UPDATE; eliminar usa DELETE. En este laboratorio se aplican a `dbo.People`.

2. **¿Por qué PeopleDB no contiene código de controles visuales?** Su responsabilidad es consultar y modificar la base. Los formularios administran los controles y muestran mensajes. Esta separación permite entender y probar las consultas sin mezclarlas con la interfaz.

3. **¿Qué diferencia existe entre una fila, un objeto People y una lista List<People>?** La fila es un registro persistido en SQL Server. El objeto People representa en memoria sus valores Id, Name y Age. List<People> reúne varios objetos para vincularlos al DataGridView. La lista no sustituye la base: se obtiene nuevamente de SQL en cada actualización.

4. **¿Qué devuelve Read() y por qué se utiliza en un ciclo?** Devuelve bool: true si avanzó a otra fila disponible y false cuando ya no hay filas. El ciclo while permite leer todas las personas. Se crea un objeto distinto en cada iteración para no reutilizar la misma referencia.

5. **¿Por qué se especifican las columnas en vez de depender de SELECT *?** Así quedan claros los datos y el orden que espera el lector: Id, Name y Age. Agregar otra columna a la tabla no cambia accidentalmente esos índices y se consultan solo los campos necesarios.

6. **¿Por qué los parámetros son preferibles a concatenar datos del usuario?** Mantienen separado el texto SQL de los valores. Un apóstrofo o un texto que parezca una instrucción se transmite como dato. También especifican el tipo y tamaño esperado, evitan problemas de conversión y previenen la inyección SQL por concatenación.

7. **¿Qué diferencia existe entre ExecuteReader y ExecuteNonQuery?** ExecuteReader devuelve un SqlDataReader para recorrer las filas de un SELECT. ExecuteNonQuery ejecuta INSERT, UPDATE o DELETE y, para estas consultas simples sin disparadores, devuelve cuántas filas fueron afectadas. Comprobar que sea 1 permite detectar un registro que desapareció antes de guardar.

8. **¿Qué función cumple WHERE Id = @Id en edición y eliminación?** Restringe la operación a la persona elegida. Id es clave primaria, por lo que como máximo coincide una fila. Sin WHERE se podrían modificar o eliminar todas las filas. El índice visual de la cuadrícula no es el ID de SQL Server.

9. **¿Por qué ShowDialog permite esperar antes de refrescar la lista?** Abre un formulario modal y la llamada espera hasta que se cierre. Entonces devuelve su DialogResult. Si es OK, el formulario principal vuelve a consultar SQL Server; si se cancela, no se guarda y no se informa una operación exitosa.

10. **¿Cómo permite int? id = null distinguir insertar de editar?** int? admite un entero o null. Si no tiene valor, se abre una persona nueva y se ejecuta Add. Si HasValue es true, se carga ese ID y se ejecuta Update. En la implementación, el constructor sin argumentos delega al constructor con null, conservando la compatibilidad con el diseñador de Visual Studio.

11. **¿Cómo demostraría que un cambio quedó guardado y no solamente en memoria?** Consultaría el ID mediante SELECT en una conexión independiente a SQL Server. Después cerraría y volvería a abrir la aplicación. Si se recuperan los mismos valores desde la base, el cambio quedó persistido. Las pruebas incluidas consultan nuevas conexiones y crean de nuevo el formulario principal.

12. **¿Una prueba manual de la aplicación es lo mismo que una prueba unitaria automatizada? Explique.** No. En una prueba manual una persona opera la interfaz y observa el resultado. Una prueba unitaria automatizada comprueba una unidad pequeña de código con entradas y resultados esperados. Las pruebas incluidas aquí son de integración, porque ejecutan formularios y consultas contra SQL Server real. Pueden repetirse automáticamente, pero no sustituyen toda la revisión visual y de interacción.